Concept Hub

---

layer: concept
type: hub
status: stable

Concept Hub

Concept Layer はVault全体の語彙体系である。
すべてのノート（theory / model / structure / pattern / case / domain）は、
Concept を共有することで接続される。

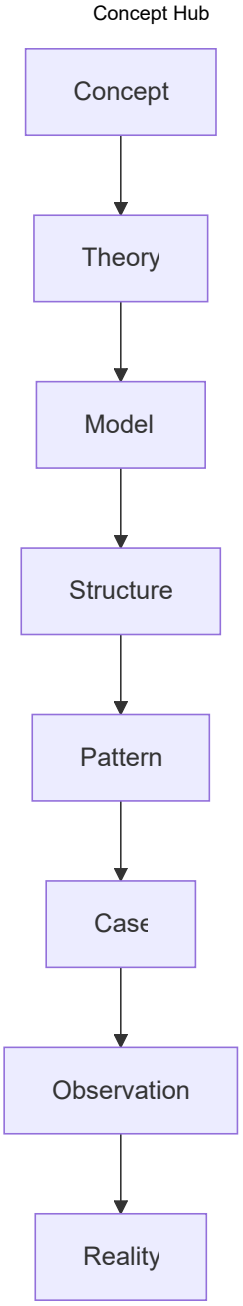
Concept は **世界を記述する最小単位の意味ノード**である。

Concept Layer の役割

Concept Layer は次の4つの機能を持つ。

1. 語彙統制（Vocabulary Control）
2. 概念関係（Concept Graph）
3. 知識統合（Knowledge Integration）
4. LLM推論安定化（Semantic Stability）

Vault における位置



Concept はすべての知識層の基盤となる。

Concept の型

Concept は5つの型に分類される。

Entity

存在する主体。

例

- Human
- Organization
- Institution
- State
- Market
- Technology

Cognitive

人間の認知。

例

- Information
- Attention
- Belief
- Knowledge
- Narrative

Social

社会関係。

例

- Norm
- Power
- Authority
- Legitimacy
- Trust

Process

行動や変化。

例

- Decision
- Learning
- Coordination
- Competition
- Conflict
- Adaptation

State

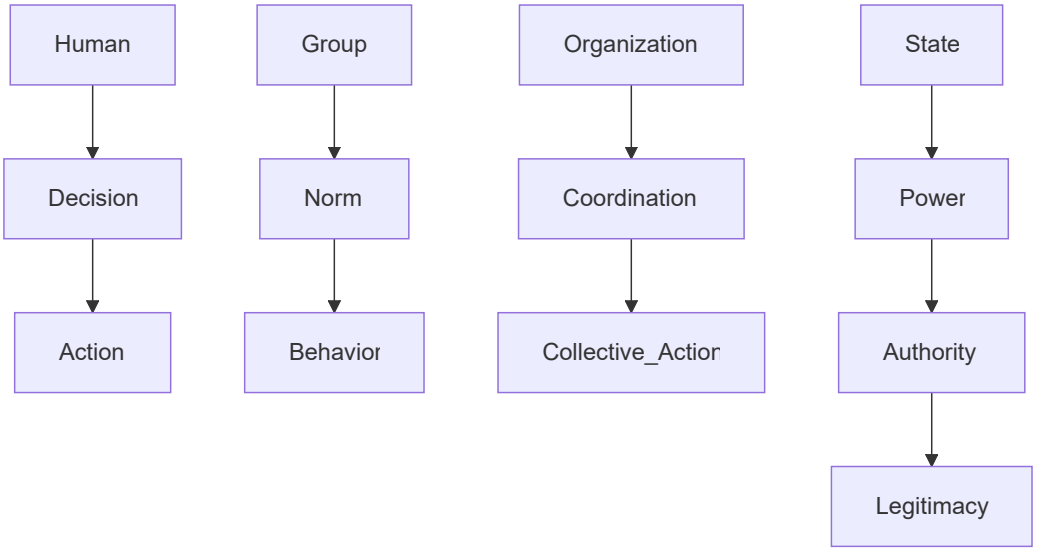
状態。

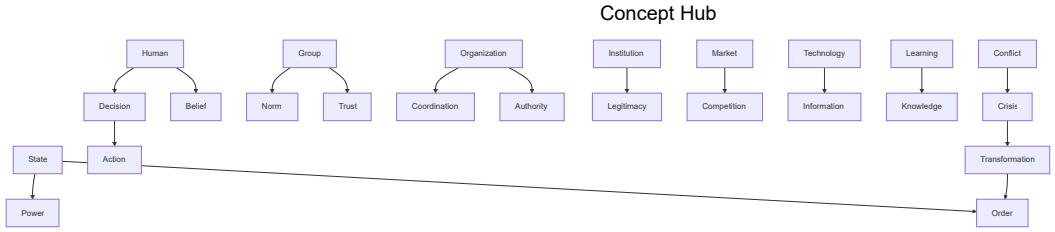
例

- Order
- Stability
- Crisis
- Collapse
- Transformation

Concept Graph

Concept は relation によって接続される。





Concept と他の層の関係

Concept → Theory

理論は概念の関係を説明する。

例

```
Power + Legitimacy → Authority
```

Concept → Model

モデルは概念を形式化したもの。

例

```
Decision → Expected Value Model
```

Concept → Structure

構造は概念の配置。

例

```
Hierarchy
Network
Market
```

Concept → Pattern

パターンは概念関係の繰り返し。

例

```
Power concentration
Bureaucratic delay
Social conformity
```

Concept → Case

事例は概念の具体例。

例

```
French Revolution
Microsoft
Tokugawa Shogunate
```

Concept の設計原則

Concept を作る際は次を守る。

1. 抽象度を一定に保つ
2. 複数分野で使える語彙にする
3. 因果関係に使える概念にする
4. 定義を簡潔にする

関連ノート

02_zettelkasten/01_ontology/Concept Types

02_zettelkasten/00_meta/concept/relations/Relation Types

Causal Relations

02_zettelkasten/04_knowledge_graph/Ontology

Concept Layer の目的

Concept Layer は

- Vault を Knowledge Graph 化する
- AI の推論精度を上げる
- 分野を越えて知識を接続する

ための基盤である。

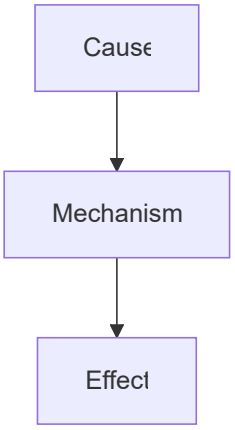
Causal Relations

Causal Relations

Causal Relations は概念・事象・構造の間に存在する因果関係を定義する。

因果関係は推論・理論・メカニズムの基盤となる。

因果関係の基本構造



Causal Relation Types

causes

ある概念・事象が直接的に別の結果を生み出す。

例

```
A causes B
```

例

```
Inflation causes social unrest
```

---

caused_by

結果側から原因を表す。

```
B caused_by A
```

---

enables

ある条件が存在することで結果が可能になる。

```
A enables B
```

例

```
Technology enables industrialization
```

---

constrains

ある要因が行動や結果を制限する。

```
A constrains B
```

例

```
Resource scarcity constrains growth
```

---

triggers

特定の出来事が結果を引き起こす引き金になる。

```
A triggers B
```

例

```
Assassination triggers war
```

---

amplifies

既存の効果を強める。

```
A amplifies B
```

例

```
Media amplifies social conflict
```

---

dampens

効果を弱める。

```
A dampens B
```

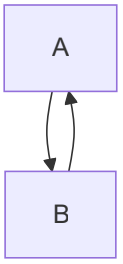
例

```
Institutions dampen political instability
```

---

feedback

結果が原因に戻る循環関係。



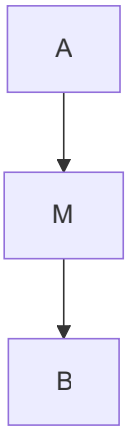
例

```
Economic growth ⇔ investment
```

---

indirect_cause

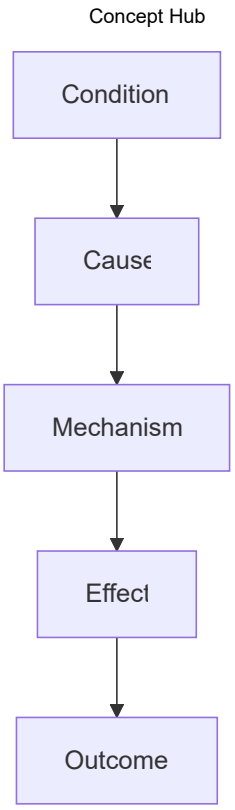
中間メカニズムを通して影響する。



例

```
Education → innovation → economic growth
```

因果関係の階層



因果分析で確認すること

因果関係を判断する際には次を確認する。

1. 時系列
2. メカニズム
3. 代替説明
4. 再現性

関連ノート

02_zettelkasten/00_meta/concept/relations/Relation Types
99_oldzettelkasten/04_knowledge_graph/Mechanism
Reasoning Pipeline
Theory

Relation Types

---

layer: concept
type: relation
status: stable

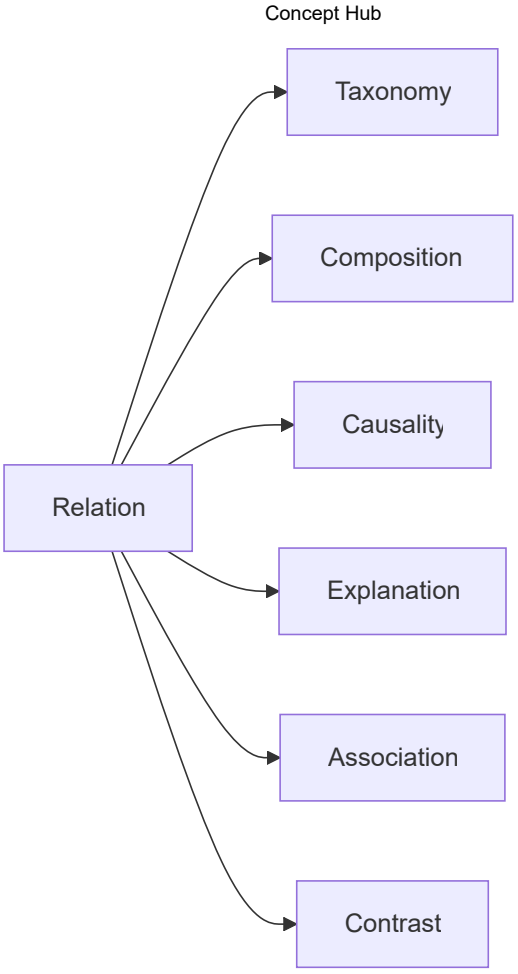
Relation Types

Relation Types は Vault における概念・理論・事象の関係を定義する。

すべてのノート間リンクは、この relation を用いて意味を明確化できる。

Relation Types は Concept Graph の基盤となる。

Relation Categories



---

1 Taxonomy Relations

概念の分類関係。

is_a

概念の上位分類。

例

```
Organization is_a Social Entity
```

---

instance_of

具体例。

例

```
Microsoft instance_of Organization
```

---

2 Composition Relations

部分関係。

part_of

構成要素。

例

```
Department part_of Organization
```

---

contains

包含関係。

例

```
Organization contains Members
```

---

3 Causal Relations

因果関係。

詳細は

Causal Relations

主な関係

- causes
- caused_by
- enables
- constrains

- triggers
- amplifies
- dampens
- feedback

例

```
Resource scarcity causes conflict
```

---

4 Explanation Relations

理論や説明関係。

explains

理論が現象を説明する。

例

```
Bureaucracy Theory explains organizational rigidity
```

---

supports

証拠が理論を支持する。

例

```
Historical case supports institutional theory
```

---

5 Association Relations

関連関係。

related_to

弱い関係。

例

```
Innovation related_to competition
```

---

influences

影響関係。

例

```
Technology influences economic growth
```

---

6 Contrast Relations

対比関係。

contrasts

対照。

例

```
Hierarchy contrasts network
```

---

alternative_to

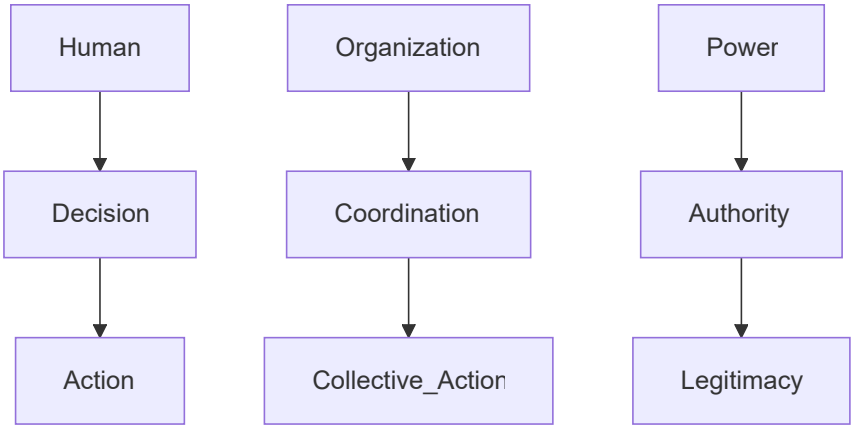
代替概念。

例

```
Market coordination alternative_to central planning
```

Relation Graph Example

Concept Hub

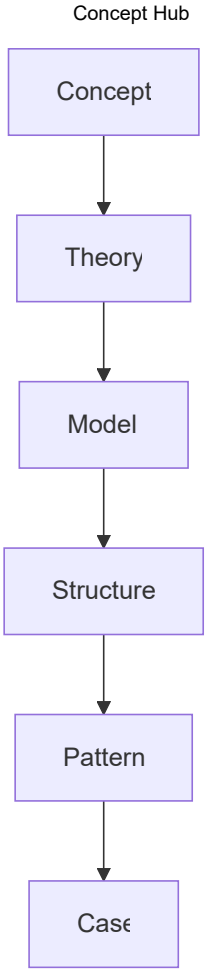


Relation Usage Rules

Vaultで relation を使う際は次を守る。

1. relation はできるだけ明示する
2. 因果関係は mechanism を伴う
3. instance_of は case に使用する
4. is_a は ontology に使用する

Relation と Knowledge Layer



relation はこれらの層を接続する。

関連ノート

99_oldzettelkasten/Concept Hub

02_zettelkasten/00_meta/concept/Concept Types

Causal Relations

02_zettelkasten/04_knowledge_graph/Ontology

Core Concept Example

Organization

複数の人間が共通目的のために協働する社会構造。

関連概念

Human
Institution
Power

関係

Organization --uses--> Coordination
Organization --creates--> Hierarchy
Organization --embedded_in--> Society

Process Concept Example

Organization

複数の人間が共通目的のために協働する社会構造。

関連概念

Human
Institution
Power

関係

Organization --uses--> Coordination
Organization --creates--> Hierarchy
Organization --embedded_in--> Society

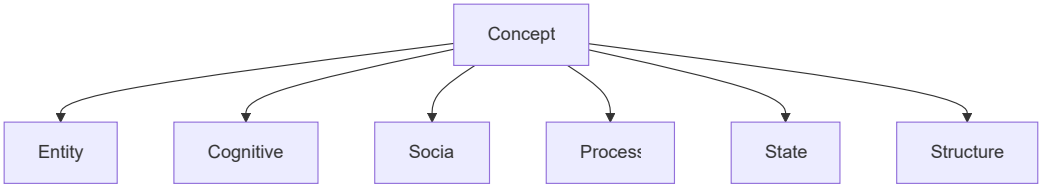
Concept Types

Concept Types

Concept Types は Vault における概念の型（カテゴリー）を定義する。

Concept は意味の最小単位であり、型を明確にすることで
Concept Graph・Theory・Model・Structure を安定させる。

Concept Type Hierarchy



---

1 Entity

Entity は存在する主体または対象。

特徴

- 行動主体または対象
- 観察可能
- 多くの場合 agent を含む

例

- Human
- Group
- Organization
- Institution
- State
- Market
- Technology

---

2 Cognitive

Cognitive は人間の認知・意味処理。

特徴

- 心理的概念
- 意味・認識・解釈

例

- Information
- Attention
- Belief
- Knowledge
- Narrative

---

3 Social

Social は社会関係や社会構造を形成する概念。

特徴

- 集団行動に関係
- 権力・規範・信頼など

例

- Norm
- Power
- Authority
- Legitimacy
- Trust

---

4 Process

Process は行動・変化・進行する現象。

特徴

- 時間的変化

- 行動または進行

例

- Decision
- Learning
- Coordination
- Competition
- Conflict
- Adaptation

---

5 State

State はシステムの状態。

特徴

- 一時的または長期状態
- システム全体の状態

例

- Order
- Stability
- Crisis
- Collapse
- Transformation

---

6 Structure

Structure は関係配置やシステム構造。

特徴

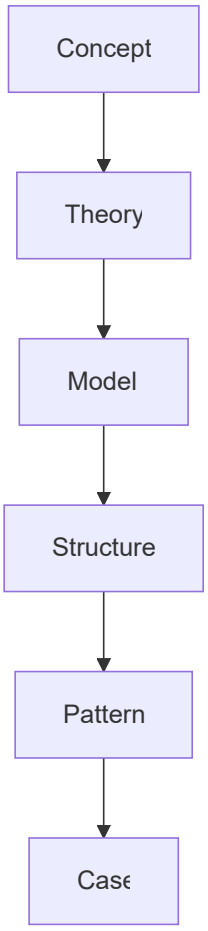
- 関係の配置
- システムの形

例

- Hierarchy
- Network

- Market
- Bureaucracy

Concept と他層の関係



Concept はすべての知識層の基盤となる。

Concept の設計原則

Concept を作る際は以下を守る。

1. 抽象度を揃える
2. 複数分野で使用できる語彙にする
3. 因果関係に使える概念にする

4. 定義を簡潔にする

関連ノート

02_zettelkasten/00_meta/concept/Concept Hub

02_zettelkasten/00_meta/concept/relations/Relation Types

Causal Relations

02_zettelkasten/04_knowledge_graph/Ontology

Concept テンプレート

Vault Knowledge Graph Architecture

Vault Knowledge Graph Architecture

このVaultは単なるノート集合ではない。

Knowledge Graph（知識グラフ）

として設計されている。

すべてのノートは
Relation Ontology に基づく関係で接続される。

基本階層

Vaultの知識は次の階層で構成される。

```
kernel
↓
mechanism
↓
pattern
↓
case
```

意味

層	役割

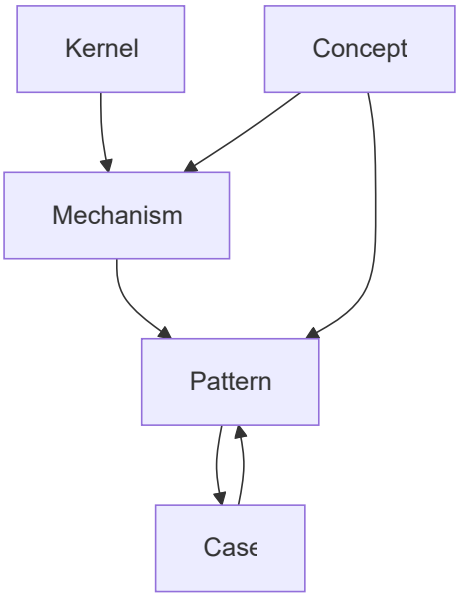
kernel|世界原理|
mechanism|作動原理|
pattern|観測パターン|
case|具体事例|

Graph Layer

Vaultは5つの知識レイヤーを持つ。

```
Kernel Layer
Mechanism Layer
Pattern Layer
Concept Layer
Case Layer
```

Layer Structure



Relation Flow

ノード間の主な関係

relation	意味
is_a	上位概念
part_of	構成
instance_of	具体例
derived_from	派生
causes	原因
explains	説明
analogous_to	類似

Knowledge Expansion

知識の拡張は以下の流れで行う。

```
Observation
↓
Case
↓
Pattern
↓
Mechanism
↓
Kernel
```

Promotion Rules

Vaultではノードが昇格する。

Case → Pattern

複数のCaseに共通構造がある場合

Patternとして抽象化する。

例

```
企業価格戦争
航空価格戦争
通信価格戦争
↓
価格戦争パターン
```

Pattern → Mechanism

パターンの原因構造が特定された場合

Mechanismに昇格する。

例

```
同調行動パターン
↓
社会的証明メカニズム
```

Mechanism → Kernel

複数分野で成立する場合

Kernelとなる。

例

```
競争メカニズム
↓
競争原理
```

Graph Traversal

Vaultの推論は以下の経路で行う。

Pattern Search

```
Question
↓
Pattern
↓
Mechanism
↓
Kernel
```

Case Reasoning

```
Question
↓
Case
↓
```

```
Pattern
↓
Mechanism
```

Concept Mapping

```
Question
↓
Concept
↓
Pattern
↓
Case
```

Knowledge Graph Types

Vaultには3種類のグラフが存在する。

1 Concept Graph

概念ネットワーク

```
Concept ⇔ Concept
```

---

2 Mechanism Graph

因果構造

```
Mechanism → Mechanism
```

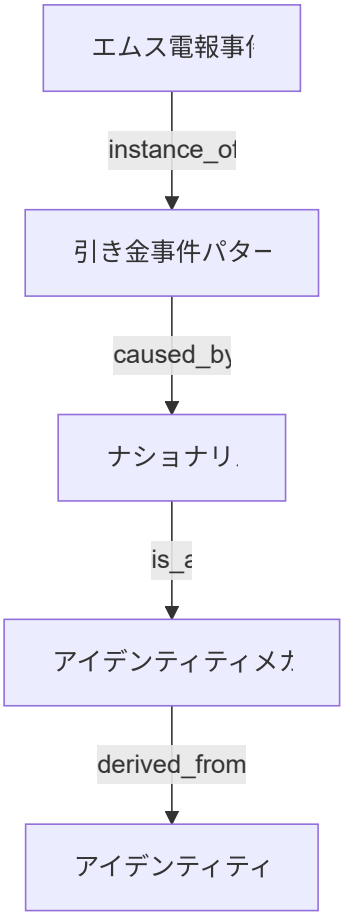
---

3 Case Graph

事例ネットワーク

```
Case → Pattern → Mechanism
```

Graph Example



Knowledge Engine

Vaultは以下のように動作する。

```
Question
↓
Relevant Concept
↓
Pattern Search
```

```
↓
Mechanism Identification
↓
Kernel Reference
↓
Answer
```

Graph Maintenance

知識グラフは以下の操作で更新される。

Add Case

新しい事例を追加。

Extract Pattern

共通構造を抽出。

Identify Mechanism

原因構造を特定。

Update Kernel

普遍原理を更新。

Design Philosophy

このVaultは

ノートの集まりではない。

思考エンジンである。

知識は

```
事例
↓
構造
↓
原理
```

へと圧縮される。

Vaultの目的は

世界の構造を抽出すること

である。

Vault Knowledge Evolution System

Vault Knowledge Evolution System

Vault Knowledge Evolution System は
このVaultにおける **知識の成長プロセス** を定義する。

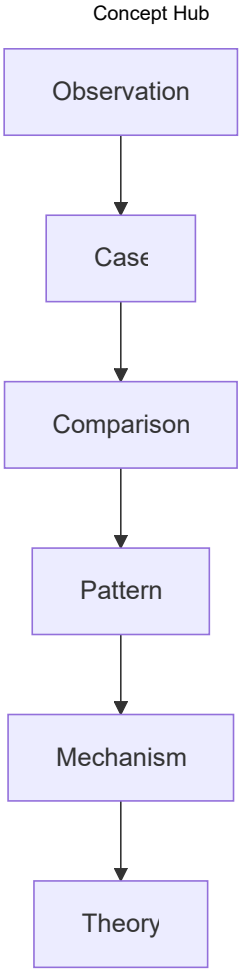
Vaultでは知識は固定された情報ではない。

知識は

観察
→ 事例
→ パターン
→ メカニズム
→ 理論

へと進化する。

知識進化の全体構造



Knowledge Levels

Vaultでは知識を5段階に分類する。

Level	内容

Observation | 観察 |
Case | 具体事例 |
Pattern | 繰り返し現れる型 |
Mechanism | 仕組み |
Theory | 抽象理論 |

Level 1 Observation

Observation は

現実の観察データである。

例

・歴史事件の記録
・企業行動
・政治発言
・制度変化

Observation は

Caseの素材

になる。

Level 2 Case

Case は

Observation を整理した **具体事例ノード**である。

例

- デイリーテレグラフ事件
- サライェヴォ事件
- ウォーターゲート事件

Caseの目的

・現象の記録
・比較の素材

Level 3 Pattern

Pattern は

複数 Case に共通する **現象の型**である。

例

- 指導者舌禍
- 革命連鎖

- 独占形成
- 責任回避

Patternの特徴

・繰り返し現れる
・抽象度が上がる

Level 4 Mechanism

Mechanism は

Pattern を生み出す **仕組み**である。

例

- 情報拡散
- ネットワーク効果
- エリート対立
- 同調圧力

Mechanism は

現象の

なぜ

を説明する。

Level 5 Theory

Theory は

Mechanism を統合した **高次抽象モデル**である。

例

- 革命理論
- 市場競争理論
- 情報社会理論

Theory は

Mechanism の集合体である。

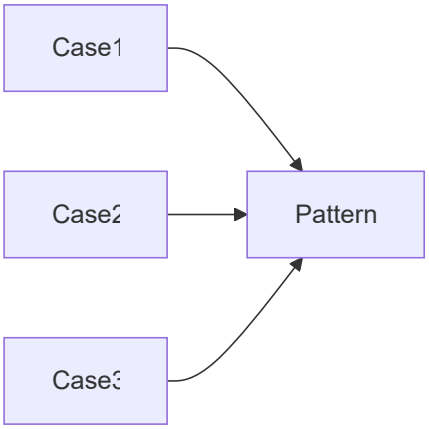
Knowledge Promotion

知識は次の条件で昇格する。

Case → Pattern

条件

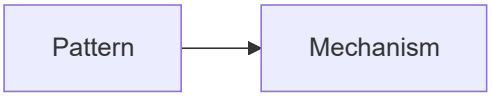
- 類似Caseが複数存在する
- 共通構造が確認できる



Pattern → Mechanism

条件

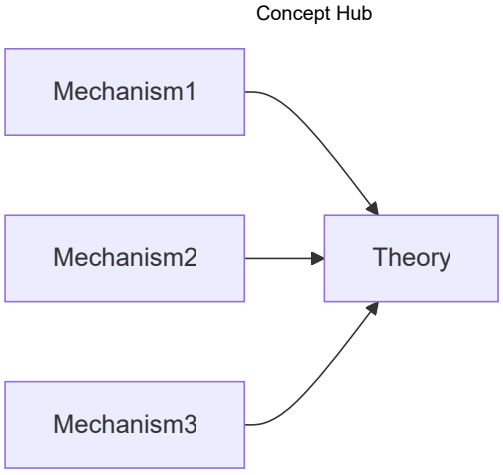
- Patternの原因構造が説明できる



Mechanism → Theory

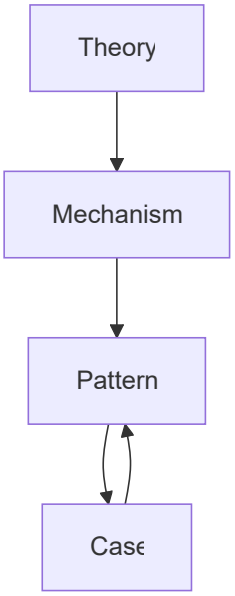
条件

- Mechanismが複数統合される



Knowledge Evolution Loop

知識は一方向ではなく循環する。



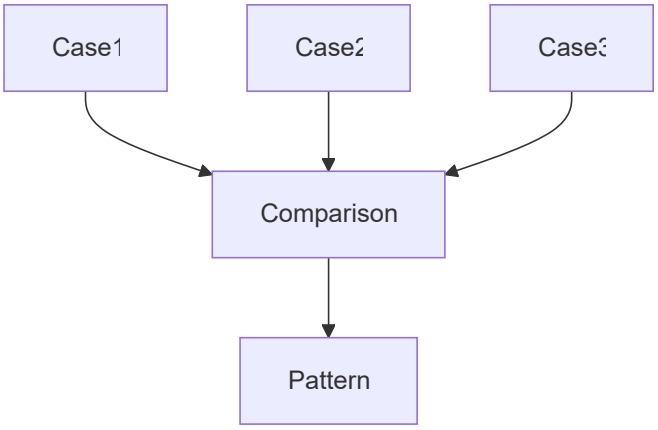
つまり

理論は新しいCaseを説明し
Caseは理論を更新する。

Pattern Extraction

Patternは

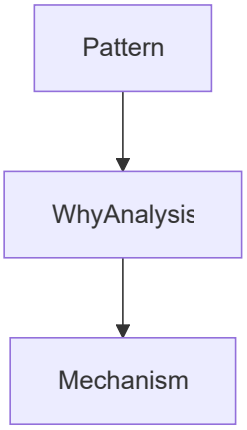
Case比較から抽出される。



Mechanism Identification

Mechanismは

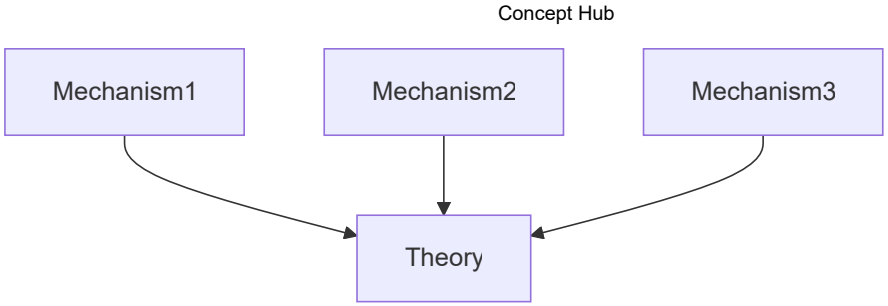
Patternの原因分析から見つかる。



Theory Construction

Theoryは

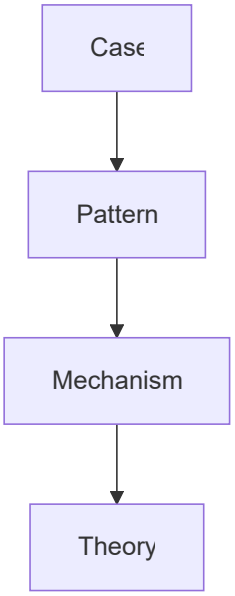
複数Mechanismの統合で作られる。



Knowledge Graph との関係

Knowledge Evolution は

Knowledge Graph 上で起きる。



つまり

Knowledge Graph は

知識の地図

Knowledge Evolution は

知識の成長

である。

Vault運用原則

Principle 1
Case を増やす。

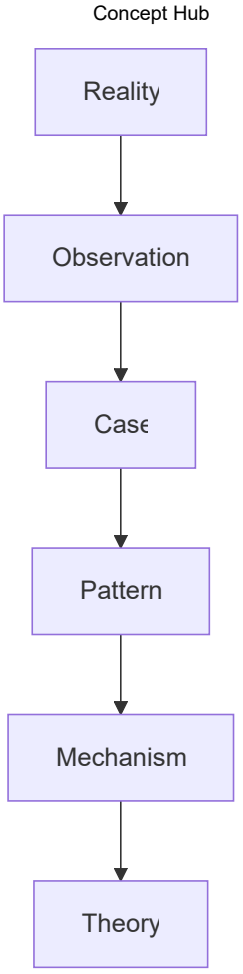
Principle 2
Comparison を行う。

Principle 3
Pattern を抽出する。

Principle 4
Mechanism を説明する。

Principle 5
Theory を構築する。

Vaultの知識生成モデル



まとめ

このVaultは

知識保存システムではない。

このVaultは

知識進化システム

である。

Caseから始まり
Theoryへ進化する。

そのプロセスを管理するのが

Vault Knowledge Evolution System である。

Unified Knowledge Classification Engine

Unified Knowledge Classification Engine

1. 目的

本ルールは、Vault内のすべての知識を以下の型に分類するための統一基準である。

- Principle（原理）
- Mechanism（法則・作動）
- Rule / Structure（律・構造）
- Pattern（パターン）
- Method（方法）
- Tool（道具）
- Framework（枠組み）
- Analysis（分析）

---

2. 全体構造

```
Reality
 ↓
Principle（なぜ）
 ↓
Mechanism（どう動く）
 ↓
Structure / Rule（どう制約される）
 ↓
Pattern（繰り返し現れる型）
 ↓
Analysis（どう読む）
 ↓
Framework（どう整理する）
 ↓
Method（どう扱う）
 ↓
Tool（何で実行する）
```

---

3. 各分類の定義と判定基準

3.1 Principle（原理）

定義

- 普遍的な説明枠組み。「なぜそうなるか」

判定

- 上位概念
- 多分野に適用可能
- 他の法則を説明する

例

- 最小作用の原理
- 希少性原理
- フィードバック原理

3.2 Mechanism（法則・作動）

定義

- 再現可能な因果過程。「どう動くか」

判定

- 入力→変化→出力
- 条件依存で再現

例

- 需要の法則
- 情報拡散
- 学習

3.3 Rule / Structure（律・構造）

定義

- 形式的制約または関係形。「どう制約されるか」

判定

- 不変条件
- 制約
- 形

例

- 保存則
- 予算制約
- ネットワーク
- 階層

3.4 Pattern（パターン）

定義

- 繰り返し現れる典型形。「よく起こる型」

判定

- 具体例から抽出される
- 再利用可能
- Mechanismの表層表現

例

- 炎上
- バブル
- サイロ化
- ロックイン

3.5 Analysis（分析）

定義

- 対象を分解・解釈する視点。「どう読むか」

判定

- 観察・理解のための視点
- 分解・比較・評価を行う

例

- 因果分析
- 構造分析
- ネットワーク分析
- フィールドワーク分析

3.6 Framework（枠組み）

定義

- 複数の要素を整理するための構造。「どう整理するか」

判定

- 複数概念の集合
- 分類や整理に使う

例

- 3C分析
- SWOT分析
- 五力分析

3.7 Method（方法）

定義

- 目的達成のための手順。「どうやるか」

判定

- 手順として再現可能
- 実行可能なプロセス

例

- 仮説検証
- 実験
- インタビュー
- フィールドワーク

3.8 Tool（道具）

定義

- Methodを実行するための手段

判定

- 実体（ソフト・ハード）
- Methodの補助

例

- Excel

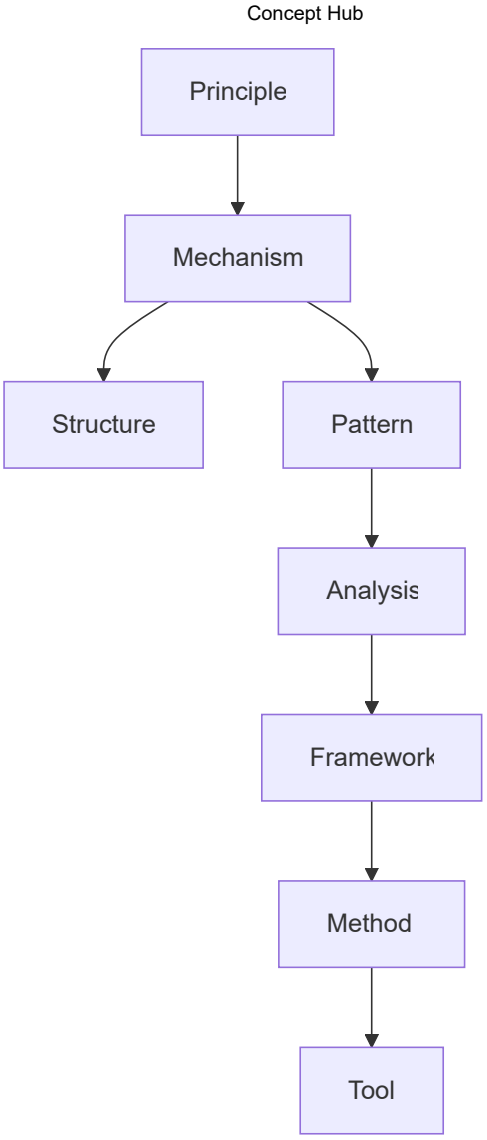
- GIS
- Notion
- Python

4. 最上位判定フロー

新規概念を見たら次の順で判定する：

- Q1 なぜを説明するか？ → Principle
- Q2 因果的に動くか？ → Mechanism
- Q3 制約・構造か？ → Rule / Structure
- Q4 繰り返しパターンか？ → Pattern
- Q5 見方・読み方か？ → Analysis
- Q6 整理枠か？ → Framework
- Q7 手順か？ → Method
- Q8 実行手段か？ → Tool

5. 典型的な対応関係



6. 同一概念の多層配置

同じ概念は複数層に現れる。

例：フィードバック

- Principle：フィードバック原理
- Mechanism：正・負フィードバック
- Structure：フィードバックループ
- Pattern：自己強化スパイラル

7. Kernelとの関係

Kernelはメタ概念であり、全レイヤーを横断する。

関係

- Principle → 抽象化元
- Mechanism → 実装形
- Structure → 形
- Pattern → 観測形

8. YAMLテンプレ

Principle

```yaml
type: principle
layer: academic
```

Mechanism

```yaml
type: mechanism
layer: academic
```

Rule / Structure

```yaml
type: rule
layer: academic
```

Pattern

```yaml
type: pattern
layer: meta
```

Analysis

```yaml
type: analysis
layer: meta
```

Framework

```yaml
type: framework
layer: meta
```

Method

```yaml
type: method
layer: meta
```

Tool

```yaml
type: tool
layer: meta
```

9. 優先順位

迷った場合：

1. Rule
2. Mechanism
3. Principle
4. Pattern
5. Analysis
6. Framework
7. Method
8. Tool

9. 最終定義

- Principle = なぜ
- Mechanism = どう動く
- Rule = 何に縛られる
- Pattern = どう現れる
- Analysis = どう読む
- Framework = どう整理する
- Method = どうやる
- Tool = 何を使ってやる

命名規則（日本語版）

命名規則（日本語版）

1. 基本方針

① 日本語で自然に読める
② 接尾語は最小限
③ 衝突時のみ明示
④ レイヤーは構造で管理

2. レイヤー別命名ルール

2.1 原理（Principle）

ルール

〜の原理

例

- フィードバック原理
- 最適化原理
- 因果原理

理由

- 抽象度が高く、明示しないと区別不能
- 学術的にも自然

2.2 メカニズム（Mechanism）

ルール

- 名詞＋動詞的表現（接尾語なし）

例

- 情報拡散
- 資源配分
- 均衡形成
- 誤差修正
- 正のフィードバック

理由

- 動きそのものなので自然文で表現可能
- 「メカニズム」は冗長

例外（衝突時）

- ～メカニズム

例：

- フィードバックメカニズム（曖昧回避用）

2.3 構造（Structure / Rule）

ルール

- 名詞（必要なら構造・制約・条件を付与）

例

- ネットワーク
- 階層構造
- フィードバックループ
- 予算制約
- 均衡条件

判断基準

状況	命名
単独で明確	ネットワーク
曖昧	ネットワーク構造

2.4 パターン（Pattern）

ルール

- 現象名（接尾語なし）

例

- ロックイン
- バブル
- 崩壊

- カスケード
- フリーライド

禁止

❌ ロックインパターン
❌ バブルパターン

2.5 分析（Analysis）

ルール

- ～分析

例

- ネットワーク分析
- 因果分析
- 構造分析
- フィールドワーク分析

2.6 フレームワーク（Framework）

ルール

- 固有名詞 or ～フレームワーク

例

- SWOT分析
- 3C
- ファイブフォース
- 価値連鎖

2.7 方法（Method）

ルール

- ～法
- ～手法

例

- 仮説検証法
- 観察法
- インタビュー法

2.8 ツール（Tool）

ルール

- 固有名詞

例

- Python
- Notion
- QGIS

3. ハブノード（最重要）

3.1 命名

- 接尾語なし

例

- フィードバック
- 均衡
- 最適化
- ネットワーク

3.2 構造

概念（ハブ）
├ 原理
├ メカニズム
├ 構造
├ パターン

3.3 例：フィードバック

フィードバック
├ フィードバック原理
├ フィードバック安定化
├ 正のフィードバック
├ フィードバックループ
├ 自己強化スパイラル

4. 衝突回避ルール

4.1 発生条件

-同一語が複数レイヤーに存在する場合

4.2 解決手順

1. ハブを作る
2. 各レイヤーに分解
3. 必要なものだけ接尾語
4. YAML補助
   命名をシンプルにする代わりに：

```
type: mechanism
layer: academic
```

5. 最短ルール

- 原理 → 必ず「〜原理」
- メカニズム → そのまま
- 構造 → 必要なら「〜構造」
- パターン → そのまま
- ハブ → そのまま

6. NG例

NG1

- フィードバック（全部入れる）

NG2

- フィードバック原理構造メカニズム

NG3

- ロックインパターン

7. 最終定義

- 名前は自然に
- 意味は構造で管理する

8. 関連

- レイヤー分類ルール
- ハブ設計ルール
- 概念分解ルール